

LLM Context Compaction Proxy

永不限, 永不停歇 · Never Full, Never Stop

中文使用指南

LLM Context Compaction Proxy Contributors

2026 年 8 月

目录

1 功能亮点	3
1.1 上下文压缩引擎	3
1.2 记忆与会话	3
1.3 Provider 与协议	3
1.4 可靠性与安全	3
1.5 MemSkill(自演进技能,V7)	4
2 支持的 Agent(OpenClaw / Claude Code / Hermes / DeepSeek Harness)	4
2.1 OpenClaw	4
2.2 Claude Code(Anthropic)	5
2.3 Hermes Agent	5
2.4 DeepSeek Harness	5
3 引言	6
3.1 为什么 README 是重要的	6
3.2 README 的主要组成部分	6
3.2.1 项目标题和描述	7
3.2.2 安装和使用说明	7
3.2.3 贡献指南和许可证	7
4 项目标题和描述	7
4.1 如何选择一个有吸引力的标题	7
4.2 如何写一个简洁明了的项目描述	7
4.2.1 描述的关键元素	8
5 安装和使用说明	8
5.1 提供详细的安装步骤	8
5.2 用户如何使用你的项目	9

5.3	提供帮助和支持	9
6	示例和代码片段	9
6.1	提供使用示例	9
6.1.1	Python SDK 示例	9
6.1.2	验证代理是否生效	10
7	项目结构和文件组织	11
7.1	说明项目的文件和目录结构	11
7.2	描述各个文件和目录的用途	11
7.2.1	主程序 (<code>compaction-proxy.py</code>)	11
7.2.2	文档目录 (<code>docs/</code>)	11
7.2.3	测试与配置	11
8	贡献指南	12
8.1	如何为项目做出贡献	12
8.1.1	提交代码的标准	12
8.2	提交问题和拉取请求的流程	12
9	许可证	13
9.1	选择合适的开源许可证	13
9.2	如何将许可证添加到你的项目中	13
9.2.1	许可证的类型和选择	13
10	联系信息和致谢	13
10.1	提供你的联系信息	13
10.2	致谢和表达对贡献者的感激	14
10.2.1	表达感激	14
10.2.2	社区文化	14

1 功能亮点

1.1 上下文压缩引擎

功能	说明
预压缩	上下文达 80% 阈值时提前触发, 避免触墙
溢出恢复	溢出错误时自动压缩并重试
增量压缩	只摘要自上次压缩后的新增消息 (CoMem)
并行压缩	大上下文分块并行压缩
并行摘要合并	将并行分块摘要合并为一份连贯摘要
双层压缩	网关层 (85% 缩减)+ Agent 层 (L1 的 50%), 三种降级路径
CJK 感知 Token 估算	精确统计中文/英文/其他字符 token 数 (两阶段)
压缩安全验证	压缩结果必须小于原文, 否则降级为激进截断
智能截断	基于角色的预算分配 (头尾拆分)
标识符保留	压缩时保持代码标识符完整
强制标识符列表	摘要中原样保留 UUID/哈希/URL/文件名
压缩缓存	30 分钟 TTL, 指令感知 salt 键
压缩项剪枝	清理上次压缩遗留的低价值产物
查询位置优化	查询置尾、工具对邻接、近期窗口重排
缓存优化排序	层哈希感知重排 + <code>cache_control</code> 断点
结构感知工具输出压缩	代码/日志/JSON 专用压缩器
手动压缩 (<code>/compact</code>)	类似 Claude Code 的 <code>/compact</code> , 可强制压缩任意会话
激进截断兜底	压缩失败或膨胀时降级为截断

1.2 记忆与会话

功能	说明
语义记忆	情节—语义双层记忆 (arXiv:2605.17625), 6 类知识槽位
跨会话记忆 + FTS5	SQLite 会话持久化 + 全文搜索
用户画像记忆	持久化用户偏好 (USER.md), 有界持久化
会话转录存档	保存/恢复原始转录以支持会话恢复
会话恢复	从转录 + 摘要恢复会话 (POST <code>/sessions/{id}/resume</code>)
背景知识注入	将近期会话知识注入压缩提示
会话上下文注入	自动向出站请求注入会话上下文
ARC 地址化引用	长工具结果替换为 ID 引用
ARC 持久化回查	ARC 条目持久化到数据库, 经 GET <code>/arc/{arc_id}</code> 回查
承诺提取 (CCL)	提取并保留目标/约束/决策/错误
LLM 记忆提取	LLM 驱动知识提取, 正则兜底
思考掩码	剥离推理内容以节省 token

密钥脱敏	自动脱敏 API Key/JWT/密码/Token
孤儿工具对清理	清理悬空的 tool_call/tool_result 对
工具对邻接修复	修复非邻接的工具调用/结果顺序

1.3 Provider 与协议

功能	说明
Provider 抽象层	通用 OpenAI/Anthropic/Gemini 兼容 (ABC)
自动 Provider 探测	依据模型名/请求头/URL 自动识别格式
OpenAI→Anthropic 请求转换	完整字段映射, 含 tool_calls → tool_use
Anthropic→OpenAI 响应转换	content/thinking/tool_use 映射回 OpenAI
SSE 流式转换	Anthropic SSE → OpenAI SSE 逐事件转换
Anthropic 仅思考重试	响应仅含空思考块时自动流式重试
独立压缩 Provider	压缩模型可使用独立上游与密钥
Gemini 密钥走请求头	x-goog-api-key header, 绝不进 URL/日志
模型注册表	67+ 已知上下文窗口的模型
透传路由	任意未匹配路径透明转发上游

1.4 可靠性与安全

功能	说明
熔断器	3 次失败 → 60 秒冷却, 三状态机
抖动检测	检测压缩循环 (3 次 / 5 条消息)→ 激进截断
端点认证	22+ 端点受保护; 无密钥时仅回环放行
并发安全	语义记忆线程锁、无竞态提示词 (参数传递)
压缩前后 Hooks	压缩前后的 HTTP webhook(可阻断)
健康与指标端点	/health、/metrics(Prometheus 风格计数器)
自适应保留轮次	对话过短时自动下调保留轮次
收缩窗口重试	每次重试递减保留轮次

1.5 MemSkill(自演进技能,V7)

功能	说明
技能注册表	CRUD + 激活生命周期 + 快照回滚 (持久化到 SQLite)
技能控制器	关键词匹配 + Gumbel-Top-K 选择
参数覆盖管线	每技能的重要性权重 + 保真乘数
DELETE 型技能管线跳过	跳过 DELETE 操作的语义提取

技能性能追踪	每技能奖励/成功统计 (/skills/{id}/performance)
技能轨迹	近期压缩轨迹 (/skills/trajectories)
技能设计器	自动设计新技能 (/skills/designer/trigger)

1.6 HTTP API 端点 (31 个)

方法	端点	说明
POST	/chat/completions(+/v1/chat/completions)	OpenAI 兼容对话 (流式与非流式)
POST	/v1/messages	Anthropic Messages API
POST	/compact	手动压缩会话
POST	/summarize	供 CompactionProvider 插件使用的纯摘要点
POST	/session	创建会话
GET	/sessions/search	FTS5 全文会话搜索
GET	/sessions/recent	近期会话
GET	/sessions/{id}	会话详情
GET	/sessions/{id}/transcript	原始转录导出
POST	/sessions/{id}/resume	恢复会话
GET/POST	/profile	用户画像读取/设置
GET/DELETE	/memory	语义记忆读取/清空
GET	/arc/{arc_id}	ARC 引用回查
GET	/skills、/skills/{id}	技能列表/详情
POST	/skills	创建技能 (草稿)
POST	/skills/{id}/activate、/deprecate	技能生命周期
POST	/skills/{id}/rollback	回滚到快照版本
GET	/skills/{id}/performance	技能奖励统计
GET	/skills/trajectories	压缩轨迹
POST	/skills/designer/trigger	触发技能设计器
GET	/health、/metrics	健康与指标
ANY	/path:.*	透传到上游

2 支持的 Agent(OpenClaw / Claude Code / Hermes / DeepSeek Harness)

代理已实测兼容 OpenClaw、Claude Code、Hermes 与 DeepSeek Harness 四大 Agent 框架。接入后上下文压缩完全由代理接管,Agent 自身不再压缩,可无限期运行。

2.1 OpenClaw

在 openclaw.json 中将模型 provider 指向代理, 并启用 v6-compaction-provider 插件:

```
{
  "models": {
    "providers": {
      "compaction-proxy": {
        "baseUrl": "http://127.0.0.1:8198",
        "api": "openai-completions",
        "models": [{ "id": "xopdeepseekv4flash" }]
      }
    }
  },
  "plugins": {
    "entries": {
      "v6-compaction-provider": {
        "enabled": true,
        "config": { "proxyUrl": "http://127.0.0.1:8198", "model": "
          xsparkx2agent" }
      }
    }
  }
}
```

已验证:OpenClaw 的 xunfei provider 已指向 127.0.0.1:8198, v6-compaction-provider 插件经 /summarize 走代理; OpenClaw 原生 compaction 已关闭 (compaction.enabled=false), 压缩完全由代理接管。

2.2 Claude Code(Anthropic)

```
export ANTHROPIC_BASE_URL=http://localhost:8198
claude
```

或在 7.claude/settings.json 中:

```
{
  "env": { "ANTHROPIC_BASE_URL": "http://localhost:8198" },
  "autoCompactEnabled": false
}
```

已验证:POST /v1/messages 经代理路由 (缺 key 返回 401 证明 Anthropic 格式识别成功);autoCompactEnabled: false 关闭 Claude Code 自带压缩。

2.3 Hermes Agent

编辑 7.hermes/config.yaml:

```
model:
  api_mode: chat_completions      # OpenAI 格式
  base_url: http://127.0.0.1:8198/v1 # 指向代理
  default: xopglm51               # 上游可用模型均可
```

已验证:Hermes 的 chat_completions 模式与代理 /v1/chat/completions 完全兼容;Hermes 自带压缩已关闭 (compression.enabled: false)。

2.4 DeepSeek Harness

编辑 \$DSH_HOME/settings.yaml(默认 7.dsh/settings.yaml):

```
llm-pi-ai:
  providers:
    xfyun-maas:
      apiKeyEnv: XF_MASS_API_KEY
      api: openai-completions      # OpenAI 兼容协议
      baseUrl: http://127.0.0.1:8198 # 指向代理
      models:
        - id: xsparkx2agent
          contextWindow: 131072
          maxTokens: 8192
```

已验证:Harness 的 xfyun-maas provider 已指向 127.0.0.1:8198;Harness 原生压缩经 cordis.patch.yml 关闭 (dsh-compaction-basic → auto: false)。

3 引言

在开源项目的世界里,一份出色的 README 文件就像是一座灯塔,指引着初次来访的开发者和用户了解项目的核心价值和使用方法。正如 Antoine de Saint-Exupéry 在《小王子》中所说:“人们只有用心去看,才能看到真实。事物的真实本质是肉眼无法看到的。”

3.1 为什么 README 是重要的

README 不仅是项目的入口,更是项目的名片。它向访问者展示了项目的目标、功能和使用方法。一份详尽、清晰的 README 能够帮助开发者快速理解项目的价值和用途,从而决定是否要使用或参与该项目。

在《代码大全》(Code Complete) 中,Steve McConnell 强调了清晰、准确的文档对于软件开发的重要性。他指出,良好的文档能够帮助开发者避免很多不必要的错误和困惑,提高工作效率。

3.2 README 的主要组成部分

一份完整的 README 通常包括项目标题、描述、安装指南、使用示例、贡献指南、许可证等部分。每一部分都是为了让读者更全面、深入地了解项目。

优点	描述
可读性	README 提供了清晰、简洁的项目信息, 帮助开发者快速了解项目。
可接入性	通过详细的安装和使用说明, 使得用户能够轻松地开始使用项目。
可维护性	为贡献者提供了明确的指南和文档, 方便项目的维护和扩展。

在《人月神话》(The Mythical Man-Month) 中, Fred Brooks 提到: “良好的文档是软件产品成功的关键。”他强调, 文档不仅是为了解释代码的功能, 更是为了传达开发者的思维和设计理念。

3.2.1 项目标题和描述

项目的标题和描述是 README 的核心部分, 它简洁明了地传达了项目的主要目的和功能。一个好的标题能够快速吸引读者的注意力, 而一个清晰的描述能够帮助读者理解项目的用途和价值。

3.2.2 安装和使用说明

这一部分提供了详细的步骤, 指导读者如何安装和使用项目。它应该包括所有必要的命令、脚本和其他相关信息, 以确保读者能够顺利地运行和使用项目。

3.2.3 贡献指南和许可证

贡献指南帮助潜在的贡献者了解如何参与项目, 而许可证则明确了项目的使用和分发条件。这两部分是开源项目不可或缺的组成部分, 它们保护了作者的权益, 也指明了用户和贡献者的权利和责任。

4 项目标题和描述

在开源项目中, 一个吸引人的标题和清晰的描述是至关重要的。它不仅是项目的第一印象, 也是决定人们是否继续阅读和探索的关键因素。

4.1 如何选择一个有吸引力的标题

一个好的标题应该是简洁、明确和具有吸引力的。它需要准确地传达项目的主要功能或特点, 同时激发读者的兴趣和好奇心。正如《代码大全》中所说: “好的代码是自解释的。”一个好的标题也应该是自解释的, 能够快速告诉读者这个项目是关于什么的。

特点	好的标题	不好的标题
简洁性	LLM Compaction Proxy	一个基于 Python 的上下文压缩代理
明确性	Context Compaction Proxy	压缩项目
吸引力	CompactionProxy: 让上下文永不超限	上下文项目

4.2 如何写一个简洁明了的项目描述

项目描述是对标题的扩展和补充, 它提供了关于项目的更多细节。一个好的项目描述应该简洁明了, 但也足够详细, 能够帮助读者快速理解项目的用途、功能和优势。

例如, 本项目 (LLM Context Compaction Proxy) 的描述可以是: “*LLM Context Compaction Proxy* 是一个透明、零配置的 *LLM API* 上下文压缩代理。它位于 *AI Agent* 与任意 *LLM* 提供商之间, 当对话上下文逼近模型 *token* 上限时, 自动压缩对话历史, 使 *Agent* 无限期运行而不触碰上下文窗口墙。”

4.2.1 描述的关键元素

- **简洁性:** 保持描述简洁, 避免冗余和复杂的术语;
- **明确性:** 清晰地表达项目的主要功能和优势;
- **具体性:** 提供具体的例子和用途, 帮助读者更好地理解项目。

一个好的描述能够让读者在短时间内快速把握项目的核心价值和功能。正如《人月神话》中所说: “在好的设计中, 复杂的事物被简化。” 一个好的项目描述也应该追求这种简洁和明确, 使读者能够快速理解和使用。

5 安装和使用说明

在开源项目中, 一个清晰、简洁的安装和使用说明是至关重要的。它不仅帮助用户快速理解如何开始, 也减少了他们在安装和使用过程中可能遇到的困惑和挫败感。

5.1 提供详细的安装步骤

首先, 我们需要提供一个详细的步骤说明, 让用户能够轻松地安装和配置项目。

1. **克隆仓库:** 从 GitHub 或其他代码托管平台克隆项目到本地:

```
git clone https://github.com/061115xhsm/llm-compaction-proxy.git
cd llm-compaction-proxy
```

2. **安装依赖:** 项目所需依赖为 `aiohttp`(Python 3.10+):

```
pip install aiohttp
```

3. 配置上游并运行: 设置上游 LLM API 地址与压缩模型, 启动代理:

```
export COMPACTION_PROXY_UPSTREAM=https://api.openai.com/v1
export COMPACTION_PROXY_MODEL=gpt-4o-mini
python3 compaction-proxy.py
```

代理监听在 localhost:8198。将 Agent 的 API 端点指向此处即可。

正如《C++ 编程思想》中所说:“代码的清晰是优秀软件的基石。”清晰的安装步骤同样是优秀文档的基石。

5.2 用户如何使用你的项目

在这一部分, 我们需要详细说明如何使用项目, 可以通过提供具体的使用场景和示例来帮助用户理解。例如, 将 Claude Code 指向代理:

```
export ANTHROPIC_BASE_URL=http://localhost:8198
claude
```

使用场景	示例代码	说明
Claude Code	<pre>export ANTHROPIC_BASE_URL=http://localhost:8198</pre>	将 Anthropic 请求路由至代理
OpenAI 兼容 Agent	<pre>export OPENAI_BASE_URL=http://localhost:8198/v1</pre>	将 OpenAI 请求路由至代理
DeepSeek	<pre>export COMPACTION_PROXY_UPSTREAM=https://api.deepseek.com/v1</pre>	以 DeepSeek 为上游

在这里, 我们可以引用《代码大全》中的一句话:“代码是给人看的, 只是恰好机器也能执行。”这意味着我们写的代码和文档, 首先是为了方便其他开发者阅读和理解。

5.3 提供帮助和支持

最后, 我们需要提供一个渠道, 让用户在遇到问题时能够获得帮助和支持。

- **问题追踪:** GitHub Issues
- **健康检查:** `curl http://localhost:8198/health`
- **文档:** 仓库内的 `README.tex` 与 `docs/technical-document.tex`

在这一部分, 我们可以引用 Donald Knuth 在《计算机程序设计艺术》中的名言:“程序是为人类读写的, 不是为机器执行的。”这强调了我们在编写代码和文档时, 应该始终考虑到人的因素, 使其易于理解和使用。

6 示例和代码片段

6.1 提供使用示例

在开源项目中, 一个清晰、直观的使用示例能帮助用户快速理解项目的用途和功能。本项目的核心用法是: 将代理置于 Agent 与 LLM 之间, 自动处理上下文压缩。

6.1.1 Python SDK 示例

以下是一个简单的示例, 展示如何通过 OpenAI SDK 经代理调用模型:

```
import openai

client = openai.OpenAI(
    base_url="http://localhost:8198/v1",
    api_key="your-key"
)

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Hello!"}],
    stream=True
)
```

在这个示例中, 我们首先创建了一个指向 localhost:8198/v1 的 OpenAI 客户端, 然后调用 chat.completions.create() 发送一条消息, 最后以流式方式读取响应。正如《C++ 编程思想》中所说:“代码是最好的教程。”这个简单的示例代码能帮助用户快速上手。

6.1.2 验证代理是否生效

```
# Health check
curl http://localhost:8198/health

# Send a test request through the proxy
curl -X POST http://localhost:8198/v1/chat/completions \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer YOUR_KEY" \
  -d '{"model": "gpt-4o-mini", "messages": [{"role": "user", "content": "Hello"}]}'
```

正如《算法导论》中所说:“一个好的算法, 应该能够应对各种情况。”这个示例不仅展示了如何使用代理, 也体现了其灵活性和实用性。

通过这种方式, 我们可以确保用户能够快速、有效地理解和使用本项目。

方面	说明
代码清晰度	代码应该是易读和易理解的, 变量和函数的命名应该清晰表达其用途和功能。
示例的实用性	示例应该是实用的, 展示项目的核心功能和特点。
用户友好性	代码和示例应该考虑用户的需求和经验, 易于上手和使用。

7 项目结构和文件组织

在开源项目中, 一个清晰、有序的项目结构和文件组织是至关重要的。它不仅能帮助开发者快速理解和参与项目, 也能使用户更容易地使用和贡献项目。

7.1 说明项目的文件和目录结构

一个项目的文件和目录结构应该是直观和自解释的。每个文件和目录都应有其特定的目的和功能。正如《代码大全》中所说: “一个好的目录结构可以帮助开发者快速地找到他们需要的信息, 从而提高生产效率。”

文件/目录	描述	用途
compaction-proxy.py	主程序	代理的全部核心逻辑
README.md / README.tex	项目说明	使用指南与文档
docs/	文档目录	技术文档与接入指南
.env.example	配置模板	环境变量配置示例
systemd/	服务模板	systemd 用户服务单元
LICENSE	许可证	MIT 许可证文本

7.2 描述各个文件和目录的用途

7.2.1 主程序 (compaction-proxy.py)

主程序包含代理的全部核心逻辑:Provider 抽象层 (OpenAI/Anthropic/Gemini)、压缩引擎、语义记忆、会话存储、技能注册表、熔断器与抖动检测等。代码按模块化组织, 每个类与函数职责单一。

7.2.2 文档目录 (docs/)

文档是项目的灵魂, 一个完备的文档目录应包含用户手册、API 文档、开发者指南等。这些文档能帮助用户和开发者更好地理解和使用项目。

正如《程序员的自我修养》中所说: “良好的文档是软件质量的保证, 也是开发者与用户沟通的桥梁。”

7.2.3 测试与配置

项目通过 `.env.example` 提供完整的配置模板, 并通过 `systemd/compaction-proxy.service` 提供系统服务化部署模板, 方便生产环境使用。

在人的思维和存在中, 组织和结构是一种常见的现象。我们的思维、行为和生活都有其内在的结构和组织。在项目管理和代码组织中, 这种结构化思维的运用能帮助我们更高效、更有条理地完成任务, 实现目标。

8 贡献指南

在开源项目中, 贡献者的参与是项目能够持续进步和发展的关键。本章节将详细介绍如何为项目做出贡献, 以及提交问题和拉取请求的流程。

8.1 如何为项目做出贡献

贡献开源项目不仅仅是代码的贡献, 还包括文档更新、问题报告、新功能建议等。正如《人月神话》中所说: “好的程序员不仅仅是写出能工作的代码, 还需要写出能维护的代码。” 这本书强调了代码质量和维护性的重要性。

1. **了解项目:** 阅读项目的文档和代码, 了解项目的目标、架构和设计原则; 参与项目的讨论和会议, 与项目成员交流;
2. **找到贡献的机会:** 查看项目的问题跟踪器, 找到你可以解决的问题; 注意项目的未来计划和里程碑, 看看哪些功能你可以贡献;
3. **贡献代码:** Fork 项目到你的 GitHub 账户; 在本地开发和测试你的代码; 提交 Pull Request。

8.1.1 提交代码的标准

- 代码必须符合项目的编码标准和风格指南;
- 提交的代码必须通过所有测试;
- 代码应该包含单元测试, 确保功能的正确性。

8.2 提交问题和拉取请求的流程

提交问题和拉取请求是开源贡献的常见形式。下面是一个简单的流程, 帮助贡献者有效地提交问题和拉取请求。

1. **提交问题:** 使用明确、具体的标题描述问题; 提供问题的详细描述, 包括重现步骤、预期行为和实际行为; 如果可能, 附加屏幕截图或动画来说明问题;
2. **提交拉取请求:** 使用清晰的标题描述拉取请求的目的; 在描述中详细说明你的更改和这些更改的必要性; 确保你的代码符合项目的编码标准和风格指南。

方面	提交问题	提交拉取请求
标题	明确、具体	清晰、描述目的
描述	详细、包含重现步骤	详细、解释更改的必要性
附加信息	屏幕截图、动画	符合编码和风格标准

在《代码大全》中，作者强调了代码质量和可读性的重要性，他说：“代码是写给人看的，顺便给机器执行。”这句话提醒我们，编写代码时不仅要考虑机器，还要考虑未来阅读和维护代码的人。

9 许可证

在开源项目中，选择合适的许可证是至关重要的一步。许可证不仅定义了其他人可以如何使用、修改和分发你的项目，还表达了项目所有者对于这些活动的态度和限制。

9.1 选择合适的开源许可证

选择许可证时，需要考虑项目的性质、目标受众以及你希望与社区的互动方式。例如，MIT 许可证 (MIT License) 允许他人自由使用、修改和分发你的代码，只要他们包含原始许可证和版权声明。

本项目采用 **MIT License**，详见仓库根目录的 `LICENSE` 文件。

正如 Richard Stallman 在《自由软件，自由社会》(Free Software, Free Society) 中所说：“自由软件是关于自由和合作。”这意味着选择一个许可证，也是在选择一个合作和分享的方式。

9.2 如何将许可证添加到你的项目中

步骤	中文描述	英文描述
1	选择合适的许可证	Choose an appropriate license
2	复制许可证文本	Copy the license text
3	创建名为 <code>LICENSE</code> 的文件	Create a file named “LICENSE”
4	将许可证文本粘贴到文件中	Paste the license text into the file
5	提交和推送更改	Commit and push the changes

在这个过程中，我们不仅是在遵循一个形式和规范，更是在建立一个开放和自由的知识共享平台。正如 Immanuel Kant 在《纯粹理性批判》(Critique of Pure Reason) 中所说：“知识的自由分享是推动人类进步的重要力量。”这也反映在我们选择和使用开源许可证的过程中。

9.2.1 许可证的类型和选择

有多种类型的开源许可证，每种都有其特定的使用场景和限制。例如，GNU General Public License (GPL) 要求任何使用、修改或分发该许可证下代码的人都必须将其更改公开，并使用相

同的许可证。

在这个世界上,知识和自由是相辅相成的。我们通过分享知识来实现自由,通过自由来推动知识的进步。在这个过程中,开源许可证扮演着桥梁和纽带的角色,连接着每一个热爱学习和分享的人。

10 联系信息和致谢

10.1 提供你的联系信息

在开源项目中,提供清晰的联系信息是非常重要的。这不仅方便其他开发者与你交流,还有助于建立一个活跃、开放和友好的社区氛围。你可以通过 GitHub Issue 与我们联系,或在项目仓库的 Discussions 中发起讨论。

例如,你可以写:“如果你有任何问题或建议,请随时通过 *GitHub Issue* 与我们联系。”

10.2 致谢和表达对贡献者的感激

每一个成功的开源项目背后,都有一个支持和贡献的社区。在这一部分,你可以表达对那些帮助项目成长的人的感激之情。正如《人类简史》中所说:“合作是人类成功的秘诀。”

10.2.1 表达感激

感谢所有为本项目贡献过代码、文档、测试与建议的开发者与用户。你的每一次提交、每一个 Issue、每一条评论,都在推动这个项目向前。

10.2.2 社区文化

建立一个积极、支持和友好的社区文化是非常重要的。你可以分享一些关于如何给予和接受帮助的经验 and 见解,以鼓励更多的人参与到项目中来。

例如,你可以写:“我们欢迎每一个人的参与和贡献,无论你的技能水平如何,都有你的一席之地。”

这种开放和包容的态度,正如《自私的基因》中所说:“生物体是由基因构建的生存机器。”在这里,每个人都是项目成功的一部分,每个人都有其独特的价值和作用。

结语

在我们的编程学习之旅中,理解是我们迈向更高层次的重要一步。然而,掌握新技能、新理念,始终需要时间和坚持。从心理学的角度看,学习往往伴随着不断的试错和调整,这就像是我们的大脑在逐渐优化其解决问题的“算法”。

这就是为什么当我们遇到错误,我们应该将其视为学习和进步的机会,而不仅仅是困扰。通过理解和解决这些问题,我们不仅可以修复当前的代码,更可以提升我们的编程能力,防止在未来的项目中犯相同的错误。

LLM Context Compaction Proxy 正是这样一段探索之旅的产物: 从研究论文中的压缩技术, 到生产环境中的工程实践; 从解决自己的上下文超限之痛, 到帮助更多开发者的 Agent 无限期运行。愿这份中文指南能成为你的灯塔, 指引你在上下文压缩的世界里, 找到属于自己的路径。

——*LLM Context Compaction Proxy Contributors*